

CAS 2 : Sauvegarde différentielle raisonné

1. Entrées :

- Dossier d'entrées des fichiers : /work/
- Dossier de sortie de sauvegardes : /backup/
- Dossier des logs : /logs/
-
- Date et heures des sauvegardes en YYYY-MM-DD_HHMM
- Seuil de versions : N versions max acceptés
- Fichiers à exclure : .tmp, .log, .cache

2. Sorties :

- Sous dossier sauvegarde : Sauvegarde_<YYYY-MM-DD>_<HHMM>
- Emplacement : /backup/
- Contenu du dossier sauvegarde : dossiers modifiés avec fichiers modifiés
- Fichier log : log_<YYYY-MM-DD>_<HHMM>.log
- Emplacement du log : /log/
- Contenu du log :
 - date et heure de l'execution (début et fin)
 - nom de la sauvegarde (Sauvegarde__)
 - nombre de fichiers copiés
 - sauvegardes éventuellement supprimées
 - Erreurs rencontrées : Alerte sauvegarde incomplète

3.Pseudo-code :

Étape 0 : Initialisation

1. Lire WORK_DIR , BACKUP_DIR, MAX_BACKUPS, LOG_FILE, EXCLUDE_PATTERNS, CRON_JOB
2. Créer WORK_DIR, BACKUP_DIR, LOG_FILE s'ils n'existent pas
3. Initialisation de la fonction setup_cron
Initialisation des variables CURRENT_BACKUP_NAME et CURRENT_BACKUP_DIR

Étape 1: Détection/création

-Si la tâche n'est pas inscrite dans cron alors Création de cette tâche

Étape 2 : Préparation du Dépôt

- Création du dossier dans CURRENT_BACKUP_DIR
- Définir BORG_REPO = CURRENT_BACKUP_DIR/borg_repo.
- Inscription dans les logs de l'action en cours
- Initialisation du repos borg
- Vérification de la création du repos borg

Étape 3 : Exécution de la Sauvegarde

- Inscription dans les log de l'action en cours
- Création de la sauvegarde de WORK_DIR dans BORG_REPO avec l'exclusion des -----
pattern EXCLUDE_PATTERNS et inscription dans les log de brog -
\$CURRENT_BACKUP_DIR/borg_log.txt
- Vérifie si la sauvegarde c'est bien passer

Étape 4 : Analyse et Nettoyage

Initialisation NUM_FILES

Inscription dans les log de l'action en cours

Initialisation BACKUP_COUNT dans le BACKUP_DIR

Si BACKUP_COUNT > MAX_BACKUPS alors :

Initialisation NUM_TO_DELETE

Inscription dans les log de l'action en cours

Suppression des sauvegarde les plus ancien

Inscription dans les log de la fin de sauvegarde CURRENT_BACKUP_DIR

4. Erreurs et limites :

Si dernière date de sauvegarde : Absente le Borg doit sauvegarder tout le dossier.

Si espace disque plein : Supprimer la compression partielle et supprimer les anciennes versions.

Si sauvegarde incomplète : supprimer la sauvegarde et recommencer le script.

5. script :

```
#!/bin/bash
```

```

# Configuration
WORK_DIR="/work/"
BACKUP_DIR="/backup/"
MAX_BACKUPS=5 # Remplace N par la valeur souhaitée
LOG_FILE="/var/log/backup_borg.log"
EXCLUDE_PATTERNS=("*.tmp" "*.log" "*.cache")
CRON_JOB="0 6 * * * /home/backup_borg.sh"

# Fonction pour logger les messages
log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE"
}

# Vérifier et configurer cron si nécessaire
setup_cron() {
    # Vérifier si la tâche cron existe déjà
    if ! crontab -l 2>/dev/null | grep -qF "$CRON_JOB"; then
        log "Configuration de la tâche cron pour les sauvegardes automatiques à 6h..."
        (crontab -l 2>/dev/null; echo "$CRON_JOB")
        log "Tâche cron ajoutée : $CRON_JOB"
    else
        log "La tâche cron est déjà configurée pour s'exécuter à 6h."
    fi
}

# Exécuter la configuration cron
setup_cron

# Créer le répertoire de sauvegarde avec le format YYYY-MM-DD_HHMM
CURRENT_BACKUP_NAME=$(date '+%Y-%m-%d_%H%M')
CURRENT_BACKUP_DIR="$BACKUP_DIR/$CURRENT_BACKUP_NAME"
mkdir -p "$CURRENT_BACKUP_DIR"

# Initialiser le dépôt Borg dans le dossier de sauvegarde actuel
BORG_REPO="$CURRENT_BACKUP_DIR/borg_repo"
log "Initialisation du dépôt Borg dans $BORG_REPO..."
borg init --encryption=none "$BORG_REPO"
if [ $? -ne 0 ]; then

```

```

log "Erreur : Impossible d'initialiser le dépôt Borg."
exit 1
fi

# Créer une sauvegarde incrémentielle avec Borg
log "Début de la sauvegarde Borg pour $WORK_DIR..."
borg create "$BORG_REPO::sauvegarde" "$WORK_DIR" \
  --exclude-from <(printf "%s\n" "${EXCLUDE_PATTERNS[@]}") \
  --stats --progress > "$CURRENT_BACKUP_DIR/borg_log.txt" 2>&1

if [ $? -ne 0 ]; then
  log "Erreur : La sauvegarde Borg a échoué. Voir
  $CURRENT_BACKUP_DIR/borg_log.txt pour plus de détails."
  rm -rf "$CURRENT_BACKUP_DIR" # Supprimer la sauvegarde incomplète
  exit 1
fi

# Compter le nombre de fichiers sauvegardés
NUM_FILES=$(borg list "$BORG_REPO::sauvegarde" | wc -l)
log "Nombre de fichiers sauvegardés : $NUM_FILES"

# Gérer la rotation des sauvegardes (supprimer les plus anciennes si >
MAX_BACKUPS)
BACKUP_COUNT=$(ls -d "$BACKUP_DIR"/2[0-9][0-9][0-9]-[0-1][0-9]-[0-3][0-
9]_[0-2][0-9][0-5][0-9] 2>/dev/null | wc -l)
if [ "$BACKUP_COUNT" -gt "$MAX_BACKUPS" ]; then
  NUM_TO_DELETE=$((BACKUP_COUNT - MAX_BACKUPS))
  log "Suppression des $NUM_TO_DELETE sauvegardes les plus anciennes..."
  ls -t "$BACKUP_DIR" | grep -E '2[0-9]{3}-[01][0-9]-[0-3][0-9]_[0-2][0-9]{3}' | tail -n
"$NUM_TO_DELETE" | while read -r old_backup; do
    rm -rf "$BACKUP_DIR/$old_backup"
    log "Suppression de la sauvegarde : $old_backup"
  done
fi

log "Sauvegarde terminée avec succès dans $CURRENT_BACKUP_DIR."

```

6. Code-python création de fichier répertoire :

```
import os
import random
import string
from datetime import datetime, timedelta

# Configuration
WORK_DIR = "/work/"
FILE_EXTENSIONS = [".txt", ".pdf", ".docx", ".xlsx", ".tmp", ".log", ".cache", ".jpg", ".png"]
NUM_FILES = 20 # Nombre total de fichiers à générer
NUM_DIRS = 3 # Nombre de sous-dossiers à créer

def generate_random_string(length=10):
    """Génère une chaîne aléatoire pour le contenu des fichiers."""
    return "".join(random.choices(string.ascii_letters + string.digits, k=length))

def generate_file(path, extension):
    """Génère un fichier avec un contenu aléatoire."""
    content = generate_random_string(100) # Contenu aléatoire de 100 caractères
    with open(path + extension, "w") as f:
        f.write(content)

def generate_work_content():
    """Génère des fichiers et des sous-dossiers dans /work/."""
    if not os.path.exists(WORK_DIR):
        os.makedirs(WORK_DIR)
        print(f"Répertoire {WORK_DIR} créé.")

    # Générer des fichiers dans le répertoire racine
    for i in range(NUM_FILES):
        file_name = f"file_{i}"
        extension = random.choice(FILE_EXTENSIONS)
        generate_file(os.path.join(WORK_DIR, file_name), extension)

    # Générer des sous-dossiers et des fichiers
    for i in range(NUM_DIRS):
        dir_name = f"subdir_{i}"
        dir_path = os.path.join(WORK_DIR, dir_name)
        os.makedirs(dir_path, exist_ok=True)
```

```
for j in range(NUM_FILES // NUM_DIRS):
    file_name = f"subfile_{i}_{j}"
    extension = random.choice(FILE_EXTENSIONS)
    generate_file(os.path.join(dir_path, file_name), extension)

print(f"Contenu généré dans {WORK_DIR} avec succès.")

if __name__ == "__main__":
    generate_work_content()
```